

Conectores Oracle y SQLite

Índice

- [Conectores Oracle y SQLite](#)
 - [Índice](#)
 - [Introducción](#)
 - [Ejercicio a realizar](#)
 - [Solución implementada](#)
 - [Recursos utilizados](#)
 - [Requisitos para lanzar el programa](#)
 - [Diagrama UML](#)

Introducción

Ejercicio realizado por Luis Miguel López Romero.

Link del repositorio: <https://github.com/Luilo-prom/luismi.git>

Ejercicio a realizar

- Se pide crear una base de datos tanto en SQLite como en Oracle con dos tablas que tengan alguna relación entre sí.
- Se pide crear un primer menú en el cual podamos elegir a qué base de datos conectarnos o hacerlo con mocks.
- Se pide un segundo menú para poder interactuar con las bases de datos, dependiendo de cuál se haya elegido.

Solución implementada

- He creado la tabla `alumno`, la cual está relacionada con la tabla `profesor` por `cod_tutor`, y ambas tablas tienen un `id` como clave primaria.
- El `id` es de tipo `Long` porque, al insertar algún alumno o profesor, no voy a conocer qué `id` están disponibles; entonces se pasa como `null` y ya las bases de datos se encargan de

auto-incrementarlo.

- Las tablas se crean al lanzar el contenedor con unos datos iniciales, pero los métodos de crear tablas están correctamente implementados también.
- He usado el modelo Vista-Controlador para tenerlo todo más organizado, reducir al máximo el acoplamiento y separar responsabilidades.
- He definido la interfaz genérica `DAO` , de modo que define los métodos CRUD para un tipo genérico; con esto, si en un futuro quisiese tener otra tabla, solo tendría que crear su DAO y hacer que extendiese de esta.
- Para las conexiones he implementado el patrón `Singleton` , con el cual consigo que solo haya una instancia de conexión en todo mi programa, haciendo así que tanto `Alumno` como `Profesor` usen la misma conexión y la base de datos no se sature con muchas conexiones.
- He añadido el método listar profesores al menú y, como consulta libre, he decidido buscar un alumno por su `id` .
- También he implementado el patrón `Factory` , por el cual, dependiendo de cómo quiera acceder, se devolverá el DAO adecuado.

Recursos utilizados

- Para aprender el patrón `Singleton` me ayudé de esta página web:
<https://www.arquitecturajava.com/ejemplo-de-java-singleton-patrones-classloaders/>.
- Para aprender a implementar bien los DAO vi algunos vídeos de esta lista de reproducción:
https://www.youtube.com/watch?v=Zu_EFN1H0U8&list=PLTd5ehIj0goMKGkcD6cB7enP0nnyYiEzw&pp=0gcJCbAEOCosWNin.
- El modelo Vista-Controlador ya lo había implementado en alguna práctica de PSP, pero también me he informado sobre él.

Requisitos para lanzar el programa

- Tener instalado Docker.
- Lanzar Docker desde consola con: `docker compose up -d` .
- Ejecutar la `App` .

Diagrama UML

